

Congestione, pianificazione (ENHSP/PDDL) e visualizzazione (SUMO)

Come funziona il sistema e cosa è stato corretto — progetto_pddl, mappa "media"

1. Panoramica del flusso

L'intero processo si svolge in quattro fasi: (1) caricamento della mappa OSM e calcolo della congestione, (2) generazione del problema PDDL con i tempi che includono la congestione, (3) pianificazione con ENHSP, (4) traduzione del piano in una simulazione SUMO. La congestione viene calcolata una sola volta, all'inizio, e influenza solo la scelta del percorso fatta da ENHSP: in SUMO non viene più ricalcolata.

2. Calcolo della congestione (al caricamento della mappa)

2.1 Le "macchine fantasma" (conteggio per strada)

Non è una vera simulazione nel tempo, ma un calcolo geometrico istantaneo, eseguito una sola volta:

- si generano 10 coppie partenza/arrivo casuali tra gli incroci selezionati, usando un generatore di numeri pseudo-casuali con seed fisso (42) — quindi sempre le stesse a parità di mappa;
- per ognuna delle 10 coppie si calcola il percorso più breve in metri (Dijkstra sulla sola distanza, senza considerare velocità o traffico);
- per ogni strada del sottografo si conta quante delle 10 "macchine" la percorrono → `vehicle_count` (da 0 a 10).

2.2 Ritardi fissi per incrocio

- `signal-delay`: 30 secondi se l'incrocio ha un semaforo nei dati OSM (`highway=traffic_signals`), altrimenti 0;
- `congestion-delay`: da 0 a 30 secondi, calcolato come (ritardo per tipo di strada in ingresso: 20s `primary`, 15s `primary_link`, 10s `secondary`, 5s `tertiary/residential`, 0s `motorway/trunk`) + (bonus densità: numero di altri incroci entro 200m, max 5, × 2 secondi). Se l'incrocio è "periferico" (oltre 600m dal centro della zona selezionata) il valore è sempre 0.

3. Dal conteggio ai tempi: il problema PDDL

Per ogni strada (arco), il conteggio delle macchine fantasma diventa un fattore moltiplicativo applicato alla velocità:

```
congestion_factor = 1 + vehicle_count / 10 (varia da 1.0 a 2.0)
effective_speed = speed / congestion_factor
arc_time = distance / effective_speed
```

Tutti questi valori — `arc-time` per ogni strada, `signal-delay` e `congestion-delay` per ogni incrocio — vengono scritti come dati ("fluents") nel file PDDL del problema, insieme alla rete stradale, alle distanze e alla posizione di partenza/arrivo.

4. Pianificazione con ENHSP

ENHSP legge il PDDL e cerca la sequenza di strade da partenza ad arrivo che minimizza `total-time`, non i metri percorsi. Ogni volta che il piano attraversa una strada si somma il relativo `arc-time` (che include già il fattore di congestione); ogni volta che attraversa un incrocio si sommano `signal-delay` e

congestion-delay. Il risultato è quindi un percorso che tiene conto sia del traffico stimato sulle strade sia dei ritardi agli incroci, e può preferire una strada più lunga ma meno "trafficata" secondo questo modello.

Il piano trovato (sequenza di nodi) viene salvato anche in un file `route_custom.json`, che fa da ponte verso SUMO.

5. Da ENHSP a SUMO: solo esecuzione, non ricalcolo

`sumo_visualize.py` legge `route_custom.json` e traduce ogni nodo del piano PDDL in un incrocio ("junction") della rete SUMO. Per individuare quale delle tre reti (piccola / media / grande) usare, prova in ordine ciascuna rete e usa la prima in cui trova sia il punto di partenza sia quello di arrivo. Tra incroci consecutivi del piano calcola con Dijkstra le strade SUMO che li collegano, ottenendo la sequenza completa di edge.

Questa sequenza diventa il percorso di un'unica auto (veicolo_enhsp) nel file `*_piano.rou.xml`. La congestione calcolata nei passaggi precedenti è già "congelata" nella scelta del percorso: in SUMO non viene rimostrata né ricalcolata, si vede solo l'auto che guida lungo il tragitto già deciso.

6. Correzioni applicate oggi (mappa "media")

- **Ordine dei veicoli in `media_piano.rou.xml`:** il veicolo `veicolo_enhsp` (`depart=1`) era elencato dopo veicoli con `depart` maggiore (3, 6, 9...). SUMO richiede l'ordine cronologico e scartava silenziosamente questo veicolo ("la macchina" non veniva mai simulata). Risolto riordinando il file: ora `veicolo_enhsp` parte a $t=1$ e arriva a destinazione a $t=447$, ben dentro il limite di 800s.
- **Percorsi rotti in `media.sumocfg`:** i campi `net-file`, `route-files` e `gui-settings-file` puntavano a una cartella di una sessione precedente, inesistente sul PC. Corretti con i percorsi reali nella cartella `progetto_pddl`.
- **Tempo massimo di simulazione fisso (800s) in `sumo_visualize.py`:** nel flusso dinamico della webapp, su percorsi lunghi (es. 3.3 km a $4 \text{ m/s} \approx 14 \text{ km/h}$) gli 800 secondi non bastavano e la simulazione si fermava a 799s con l'auto ancora in viaggio. Ora il valore di `<end>` viene calcolato dinamicamente dalla lunghezza reale del percorso del piano (con un margine di sicurezza), con minimo 800s. Per il percorso generato oggi il limite è ora 1200s e l'auto arriva a destinazione a $t=939$.